

Información del sistema en Linux

ADMINISTRACIÓN DE SISTEMAS OPERATIVOS

Que es un proceso

- Un **proceso** en un sistema operativo es una instancia de un programa en ejecución. Es cualquier programa, comando o aplicación que se está ejecutando activamente en el sistema.
- Los procesos juegan un papel fundamental en cómo el sistema operativo organiza y gestiona la ejecución de programas y la asignación de recursos.

Características de un proceso

- **Identificador de proceso (PID):** Cada proceso en ejecución tiene un identificador único llamado **PID (Process ID)**. Este número permite al sistema operativo y a las herramientas de administración identificar y manejar el proceso.

Características de un proceso

Recursos asignados: Cuando un proceso se crea, el sistema operativo asigna varios recursos, como tiempo de CPU, memoria, archivos y dispositivos de E/S (Entrada/Salida), necesarios para su ejecución. Algunos de estos recursos son

- **Memoria:** Se asigna memoria en la RAM para que el proceso almacene datos y ejecute instrucciones.
- **CPU:** El proceso recibe tiempo de la CPU para realizar sus tareas.
- **Archivos:** Puede acceder o manipular archivos en el sistema.
- **Dispositivos de E/S:** Los procesos pueden interactuar con dispositivos como impresoras, discos, etc.

Características de un proceso

Estados del proceso: Durante su ciclo de vida, un proceso puede estar en diferentes estados, como:

- **Nuevo:** El proceso está en proceso de creación.
- **Listo:** El proceso está esperando ser ejecutado por la CPU.
- **En ejecución:** El proceso está siendo ejecutado por la CPU.
- **Bloqueado:** El proceso está esperando que un evento externo (como una entrada de datos o acceso a recursos) ocurra.
- **Terminado:** El proceso ha finalizado su ejecución.

Características de un proceso

Administración de procesos: El núcleo del sistema operativo (o **kernel**) es el encargado de la administración de los procesos.

- **Creación de procesos:** Cuando un programa se ejecuta, el sistema operativo crea un proceso asociado a ese programa.
- **Planificación:** El sistema operativo decide qué procesos deben ejecutarse y en qué momento, basándose en algoritmos de planificación.
- **Sincronización:** Se gestiona la interacción entre procesos, especialmente si comparten recursos.
- **Terminación de procesos:** El sistema libera los recursos cuando el proceso termina.

Ps aux

Muestra una lista de los procesos en ejecución

```
(kali@kali)-[~]
$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.2	0.1	22292	13180	?	Ss	04:17	0:01	/sbin/init splash
root	2	0.0	0.0	0	0	?	S	04:17	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	04:17	0:00	[pool_workqueue_release]
root	4	0.0	0.0	0	0	?	I<	04:17	0:00	[kworker/R-rcu_g]
root	5	0.0	0.0	0	0	?	I<	04:17	0:00	[kworker/R-rcu_p]
root	6	0.0	0.0	0	0	?	I<	04:17	0:00	[kworker/R-slub_]
root	7	0.0	0.0	0	0	?	I<	04:17	0:00	[kworker/R-netns]
root	8	0.0	0.0	0	0	?	I	04:17	0:00	[kworker/0:0-events]
root	9	0.0	0.0	0	0	?	I<	04:17	0:00	[kworker/0:0H-events_highpri]

top

- Muestra una vista en tiempo real de los procesos que están usando más recursos

```
top - 04:36:47 up 19 min,  2 users,  load average: 0.00, 0.02, 0.00
Tasks: 249 total,   1 running, 248 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.7 us,  0.8 sy,  0.0 ni, 98.3 id,  0.0 wa,  0.0 hi,  0.1 si,  0.0 st
MiB Mem :  7939.9 total,  6885.1 free,   847.3 used,   445.6 buff/cache
MiB Swap:  1024.0 total,  1024.0 free,    0.0 used.  7092.6 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1026	kali	9	-11	177176	15552	10304	S	36.1	0.2	0:01.54	pipewire
1336	kali	20	0	214804	40400	29664	S	24.8	0.5	0:03.59	vmtoolsd
706	root	20	0	243676	8832	7552	S	22.3	0.1	0:03.12	vmtoolsd
869	root	20	0	399456	117176	54992	S	3.2	1.4	0:13.81	Xorg
274	root	20	0	0	0	0	I	0.6	0.0	0:00.47	kworker/7:2-mm_percpu_wq
1030	kali	9	-11	165500	10060	7632	S	0.3	0.1	0:00.05	pipewire-pulse
1049	kali	20	0	337056	23940	17024	S	0.3	0.3	0:00.29	xfce4-session

kill

- Se utiliza para finalizar un proceso dado su PID.

```
(kali@kali)-[~]  
$ kill 1339
```

```
MiB Mem : 7939.9 total, 6879.7 free, 843.0 used, 456.0 buff/cache  
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used. 7096.9 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
869	root	20	0	399456	117520	55208	S	4.3	1.4	0:16.96	Xorg
1256	kali	20	0	456620	98200	85148	S	1.0	1.2	0:02.08	qterminal
1146	kali	20	0	1856180	120924	80860	S	0.7	1.5	0:04.92	xfwm4
1210	kali	20	0	337684	27112	20136	S	0.3	0.3	0:02.68	panel-15-genmon
1	root	20	0	22292	13180	10108	S	0.0	0.2	0:01.70	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.02	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_g
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_p
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-slub_
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_g

Procesos en Sistemas Operativos: Tipos, Estados y Estructuras

- Un **proceso** es una unidad fundamental en los sistemas operativos que representa la ejecución de un programa o una aplicación. El proceso requiere recursos del sistema, como CPU, memoria y acceso a dispositivos, para poder funcionar.

Tipos de Procesos

Procesos interactivos o trabajos de shell:

- Son los procesos que los usuarios inician manualmente desde una línea de comandos o terminal.
- Están asociados con la **shell** (como Bash en Linux) en la que se ejecutan.
- Son interactivos porque requieren la intervención del usuario.

Tipos de Procesos

- **Demonios:**
- Son procesos que se ejecutan en segundo plano, proporcionando servicios sin interacción directa con los usuarios.
- Normalmente, se inician durante el arranque del sistema y continúan en ejecución indefinidamente, hasta que se reinician o finalizan explícitamente.
- Los demonios suelen tener nombres que terminan en "d"
- Sshd maneja las conexiones SSH al sistema.

Tipos de Procesos

- **Subprocesos del núcleo:**
- Estos procesos son parte del **kernel** del sistema operativo y realizan tareas críticas.
- No son gestionables con herramientas comunes para la administración de procesos como ps y top
- Son vitales para el funcionamiento del sistema y administran tareas de bajo nivel como la gestión de memoria o interrupciones de hardware.

Estados de un Proceso

- **En ejecución (Running):**
 - El proceso está siendo ejecutado activamente por la CPU.
 - Solo un proceso (o más si se tiene un procesador multinúcleo) puede estar en este estado en cualquier momento.
- **Durmiendo (Sleeping):**
 - El proceso está inactivo, esperando que ocurra un evento (como la entrada/salida de datos o la disponibilidad de un recurso).
 - Hay dos subtipos:
 - **Durmiendo interrumpible:** Puede ser interrumpido por una señal.
 - **Durmiendo no interrumpible:** No puede ser interrumpido hasta que el evento específico al que espera ocurra.

Estados de un Proceso

- **En espera (Waiting):**
 - El proceso está esperando a que otro recurso del sistema esté disponible o que se complete una operación.
 - Este estado es esencial para procesos que dependen de la disponibilidad de algún recurso externo.
- **Detenido (Stopped):**
 - El proceso ha sido detenido manualmente por una señal o esta siendo depurado

Estados de un Proceso

- **Zombie:**
- El proceso ha terminado su ejecución, pero su entrada en la tabla de procesos no ha sido liberada porque el proceso padre aún no ha leído su estado de salida.
- Los procesos zombie son inofensivos en pequeñas cantidades, pero si se acumulan, pueden consumir recursos del sistema innecesariamente.

Estados de un Proceso

Columna STAT (Estado del proceso)

- **R (Running)**: El proceso está en ejecución o listo para ejecutarse en la CPU.
- **S (Sleeping)**: El proceso está durmiendo, esperando algún evento (por ejemplo, entrada/salida).
- **I (Idle)**: El proceso está inactivo, sin realizar trabajo en ese momento.
- **T (Stopped/Traced)**: El proceso ha sido detenido (por ejemplo, mediante señales como SIGSTOP) o está siendo rastreado por un depurador.
- **Z (Zombie)**: El proceso ha terminado, pero su entrada aún no ha sido eliminada porque el proceso padre no ha leído su estado de salida.
- **D (Uninterruptible Sleep)**: El proceso está esperando una operación de entrada/salida que no puede ser interrumpida.

```
ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.1	0.1	23240	14204	?	Ss	04:17	0:02	/sbin/init splash
root	2	0.0	0.0	0	0	?	S	04:17	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	04:17	0:00	[pool_workqueue_release]
root	4	0.0	0.0	0	0	?	I<	04:17	0:00	[kworker/R-rcu_g]
root	5	0.0	0.0	0	0	?	I<	04:17	0:00	[kworker/R-rcu_p]
root	6	0.0	0.0	0	0	?	I<	04:17	0:00	[kworker/R-slub_]
root	7	0.0	0.0	0	0	?	I<	04:17	0:00	[kworker/R-netns]
root	9	0.0	0.0	0	0	?	I<	04:17	0:00	[kworker/0:0H-events_highpri]
root	11	0.0	0.0	0	0	?	I	04:17	0:00	[kworker/u64:0-floppy]
root	12	0.0	0.0	0	0	?	I<	04:17	0:00	[kworker/R-mm_pe]
root	13	0.0	0.0	0	0	?	I	04:17	0:00	[rcu_tasks_kthread]
root	14	0.0	0.0	0	0	?	I	04:17	0:00	[rcu_tasks_rude_kthread]
root	15	0.0	0.0	0	0	?	I	04:17	0:00	[rcu_tasks_trace_kthread]
root	16	0.0	0.0	0	0	?	S	04:17	0:00	[ksoftirqd/0]
root	17	0.0	0.0	0	0	?	I	04:17	0:00	[rcu_preempt]
root	18	0.0	0.0	0	0	?	S	04:17	0:00	[migration/0]
root	19	0.0	0.0	0	0	?	S	04:17	0:00	[idle_inject/0]
root	20	0.0	0.0	0	0	?	S	04:17	0:00	[cpuhp/0]
root	21	0.0	0.0	0	0	?	S	04:17	0:00	[cpuhp/1]
root	22	0.0	0.0	0	0	?	S	04:17	0:00	[idle_inject/1]
root	23	0.0	0.0	0	0	?	S	04:17	0:00	[migration/1]
root	24	0.0	0.0	0	0	?	S	04:17	0:00	[ksoftirqd/1]
root	25	0.0	0.0	0	0	?	I	04:17	0:00	[kworker/1:0-cgroup_destroy]
root	26	0.0	0.0	0	0	?	I<	04:17	0:00	[kworker/1:0H-events_highpri]
root	27	0.0	0.0	0	0	?	S	04:17	0:00	[cpuhp/2]
root	28	0.0	0.0	0	0	?	S	04:17	0:00	[idle_inject/2]

Estados de un Proceso

`ps aux | grep 'T'`

Este comando filtra los procesos en estado parado.

```
(kali㉿kali)-[~]  
$ ps aux | grep 'T'
```

kali	9767	0.0	0.0	9216	5248	pts/0	T	04:34	0:00	top
kali	12317	0.0	0.0	9216	5248	pts/0	T	04:39	0:00	top
kali	14286	0.0	0.0	10824	3956	pts/0	T	04:43	0:00	/us
kali	14287	0.0	0.2	33152	20480	pts/0	T	04:43	0:00	/us
kali	16124	0.0	0.0	9216	5248	pts/0	T	04:47	0:00	top
kali	27494	0.0	0.0	6356	2176	pts/0	S+	05:09	0:00	gre

Estados de un Proceso

Para ver solo los procesos durmiendo
`ps -e -o pid,stat,comm | grep 'S'`

```
(kali㉿kali)-[~]  
$ ps -e -o pid,stat,comm | grep 'S'  
  
    2 S      kthreadd  
    3 S      pool_workqueue_release  
   16 S      ksoftirqd/0  
   18 S      migration/0  
   19 S      idle_inject/0  
   20 S      cpuhp/0  
   21 S      cpuhp/1  
   22 S      idle_inject/1  
   23 S      migration/1
```

- Para ver solo los procesos parados
- `ps -e -o pid,stat,comm | grep 'T'`

```
(kali㉿kali)-[~]  
$ ps -e -o pid,stat,comm | grep 'T'  
  
  9767 T      top  
 12317 T      top  
 14286 T      zsh  
 14287 T      command-not-fou  
 16124 T      top  
 33793 T      top  
  
(kali㉿kali)-[~]
```


Estructura de un Proceso

- **PID (Process Identifier):**

- Cada proceso tiene un **PID** único, que es su identificador en el sistema. El PID es asignado por el kernel en el momento de la creación del proceso y es esencial para la gestión y comunicación entre procesos.

- **PPID (Parent Process Identifier):**

- Es el identificador del **proceso padre**, que es el proceso que creó al proceso actual.
- Los procesos pueden tener múltiples hijos, formando una estructura jerárquica.

Estructura de un Proceso

- **Tabla de procesos:**
 - Es una estructura interna que el sistema operativo utiliza para almacenar información sobre todos los procesos activos, como el estado del proceso, su prioridad, los recursos asignados, etc.
- **Espacio de direcciones:**
 - El proceso tiene su propio espacio de direcciones de memoria, donde se almacenan el código del programa, los datos y el estado de ejecución (como los registros de CPU).
- **Contexto del proceso:**
 - Cuando un proceso se pausa y otro se ejecuta, el sistema guarda el **contexto** del proceso en pausa, lo que incluye el estado de los registros de la CPU, la pila, el contador de programa, etc.
 - Al volver a ejecutar el proceso, este contexto se restaura para que continúe desde donde se detuvo.

Gestión de Procesos mediante Señales

- Los procesos pueden ser gestionados a través de **señales**, que son un mecanismo para enviar comandos a los procesos para que realicen ciertas acciones:
- Kill: Se usa el comando kill seguido del PID del proceso.
- SIGSTOP: Usar SIGSTOP para detener un proceso temporalmente y SIGCONT para reanudarlo.
 - kill -SIGSTOP 12345 y kill -SIGCONT 12345

Hilos de Ejecucion

- Los hilos de ejecución (threads) en Linux permiten que un proceso realice múltiples tareas en paralelo. Cada hilo puede ejecutar una parte del código del proceso y compartir recursos como la memoria. Esta característica es especialmente útil en sistemas con múltiples núcleos de CPU, ya que permite que los hilos se ejecuten simultáneamente en diferentes núcleos, mejorando el rendimiento general de la aplicación.

Características de los Hilos en Linux

- **Manejo de hilos:** Aunque Linux ofrece soporte para hilos de ejecución, no gestiona directamente su creación o sincronización. Este trabajo recae en los programadores de las aplicaciones multihilo. Es decir, el programador es responsable de la creación y manejo de los hilos mediante la implementación de las librerías adecuadas, como **POSIX threads (pthreads)**.
- **Compartición de recursos:** Todos los hilos dentro de un proceso comparten el mismo espacio de memoria, archivos abiertos, y otras estructuras de datos. Esto facilita la comunicación entre hilos pero también introduce desafíos como la gestión adecuada de la concurrencia y la sincronización para evitar problemas.

Tipos de Procesos en Segundo Plano

Subprocesos del Kernel (Kernel threads:

- Son parte del propio núcleo de Linux y se ejecutan en segundo plano. Estos subprocesos se encargan de diversas tareas del sistema operativo, como la gestión de dispositivos o el manejo de interrupciones.
- Cada uno de estos subprocesos tiene su propio PID (Process ID). En las herramientas de monitoreo de procesos, los subprocesos del kernel suelen aparecer con nombres entre corchetes, como [kworker/0:0], lo que facilita su identificación.

Tipos de Procesos en Segundo Plano

Procesos Daemon:

- Son procesos en segundo plano diseñados para realizar tareas de soporte en el sistema. Estos procesos no interactúan directamente con el usuario, pero proporcionan servicios esenciales como el manejo de redes o el registro de eventos.
- A diferencia de los subprocesos del kernel, los **daemons** son procesos de usuario que se inician al arranque del sistema o cuando se necesitan ciertos servicios.
- Un ejemplo común de daemon es **sshd**, que maneja conexiones SSH.

Tipos de Procesos en Segundo Plano

- Ver procesos y hilos:

- Ps -eLf

```
(kali㉿kali)-[~]  
$ ps -eLf
```

UID	PID	PPID	LWP	C	NLWP	STIME	TTY	TIME	CMD
root	1	0	1	0	1	04:17	?	00:00:02	/sbin/init splash
root	2	0	2	0	1	04:17	?	00:00:00	[kthreadd]
root	3	2	3	0	1	04:17	?	00:00:00	[pool_workqueue_release]
root	4	2	4	0	1	04:17	?	00:00:00	[kworker/R-rcu_g]
root	5	2	5	0	1	04:17	?	00:00:00	[kworker/R-rcu_p]
root	6	2	6	0	1	04:17	?	00:00:00	[kworker/R-slub_]
root	7	2	7	0	1	04:17	?	00:00:00	[kworker/R-netns]
root	9	2	9	0	1	04:17	?	00:00:00	[kworker/0:0H-events_highpri]
root	11	2	11	0	1	04:17	?	00:00:00	[kworker/u64:0-floppy]
root	12	2	12	0	1	04:17	?	00:00:00	[kworker/R-mm_pe]
root	13	2	13	0	1	04:17	?	00:00:00	[rcu_tasks_kthread]
root	14	2	14	0	1	04:17	?	00:00:00	[rcu_tasks_rude_kthread]
root	15	2	15	0	1	04:17	?	00:00:00	[rcu_tasks_trace_kthread]
root	16	2	16	0	1	04:17	?	00:00:00	[ksoftirqd/0]

Transiciones de estados

- En Linux, los procesos tienen una jerarquía, y la relación entre procesos padre e hijos es importante, especialmente cuando se gestionan mediante señales. Las transiciones de estados de los procesos (como iniciarse, detenerse o morir) dependen de estas señales, que son mensajes que se envían para realizar alguna acción en un proceso.

Señales Comunes en Linux

- **SIGTERM (15):** Señal para terminar un proceso de forma controlada. El proceso puede interceptarla, limpiar recursos y finalizar de manera segura. Esto es útil cuando queremos que un proceso finalice de forma ordenada.
- Comando: Kill
- Ejemplo: Kill (pid)

Señales Comunes en Linux

- **SIGKILL (9):** Señal para **forzar** la terminación de un proceso. A diferencia de SIGTERM, no puede ser ignorada por el proceso, y este se detendrá de inmediato. Sin embargo, esto puede provocar que se pierdan datos no guardados, ya que el proceso no tiene oportunidad de realizar una limpieza antes de morir.
- Comando: `kill -9`
- Ejemplo: `Kill -9 1474`

Señales Comunes en Linux

- **SIGHUP (1)**: Señal utilizada tradicionalmente para "colgar" o finalizar un proceso. Sin embargo, en muchos servicios de Linux, como **servidores web o de bases de datos**, esta señal se usa para reiniciar un proceso y recargar sus archivos de configuración sin tener que finalizarlo completamente.
- Comando: `kill -SIGHUP PID]`
- Ejemplo: `Kill -SIGHUP 1474`

Prioridad de procesos

- En Linux, la gestión de prioridades es crucial para determinar qué procesos recibirán más o menos tiempo de CPU, lo que influye directamente en el rendimiento del sistema. Cada proceso tiene un **valor de prioridad** y un **valor de nice**, que determinan cómo el kernel asigna recursos entre los diferentes procesos.

Prioridades de procesos

- Los valores de prioridad en Linux están organizados en un rango de 0 a 39. Cuanto más bajo es el valor de prioridad, mayor será la prioridad que recibe el proceso en términos de asignación de CPU.
- El valor por defecto que se asigna a un proceso cuando se crea es 20, lo que corresponde a una prioridad "neutral", es decir, ni particularmente alta ni baja.

Cambiar la prioridad de un proceso

- Nice: cambia la prioridad de un recurso ganando así más recursos y tiempo del cpu
 - Ejemplo: `nice -n -5 (nombre del proceso)`
- Renice: si el proceso ya está en ejecución y queremos mejorar la prioridad usamos este comando.
 - `Sudo renice -n 10 -p (nombre del proceso)`

Cambiar la prioridad de un proceso

```
$ top
top - 06:14:16 up 1:56, 2 users, load average: 0.00, 0.01, 0.00
Tasks: 259 total, 2 running, 248 sleeping, 9 stopped, 0 zombie
%Cpu(s): 0.5 us, 0.4 sy, 0.0 ni, 99.1 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7939.9 total, 6725.4 free, 968.1 used, 498.6 buff/cache
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used. 6971.8 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
  869 root        20   0   404804   119220   55628 S   2.3   1.5   1:36.26 Xorg
 1336 kali        20   0   220464    46236   29792 S   1.3   0.6   0:35.10 vmtoolsd
 1256 kali        20   0   460416    98296   84988 S   0.7   1.2   0:11.40 qterminal
60200 kali      20   0    9216    5248    3072 R   0.7   0.1   0:00.08 top
   66 root        20   0         0         0         0 I   0.3   0.0   0:00.02 kworker/u68:0-flush-8
 786 root        20   0    217188    8868    7552 S   0.0   0.1   0:01.70
```


Identificación de los procesos del sistema

- Comando ps
- El comando ps (process status) te permite ver los procesos que están corriendo en el sistema
- Comando: ps -e

```
—(kali㉿kali)-[~]  
$ ps -e  
  PID TTY          TIME CMD  
    1 ?           00:00:02 systemd  
    2 ?           00:00:00 kthreadd  
    3 ?           00:00:00 pool_workqueue_release  
    4 ?           00:00:00 kworker/R-rcu_g  
    5 ?           00:00:00 kworker/R-rcu_p  
    6 ?           00:00:00 kworker/R-slub_  
    7 ?           00:00:00 kworker/R-netns  
    9 ?           00:00:00 kworker/0:0H-events_highpri  
   11 ?           00:00:00 kworker/u64:0-floppy  
   12 ?           00:00:00 kworker/R-mm_pe  
   13 ?           00:00:00 rcu_tasks_kthread  
   14 ?           00:00:00 rcu_tasks_rude_kthread  
   15 ?           00:00:00 rcu_tasks_trace_kthread
```


Identificación de los procesos del sistema

- Listar procesos con paginación:
 - `ps -e | more -20`
- Buscar un proceso específico por nombre
 - `ps -el | grep (nombre del proceso)`
- Buscar un proceso específico por PID
 - `ps -e | grep <número_de_PID>`

Identificación de los procesos del sistema

- Comando pgrep.
- El comando pgrep es útil cuando quieres obtener el PID de un proceso por su nombre

- `pgrep ssh`

```
(kali@kali)-[~]  
$ pgrep ssh  
  
868  
1103
```


Identificación de los procesos del sistema

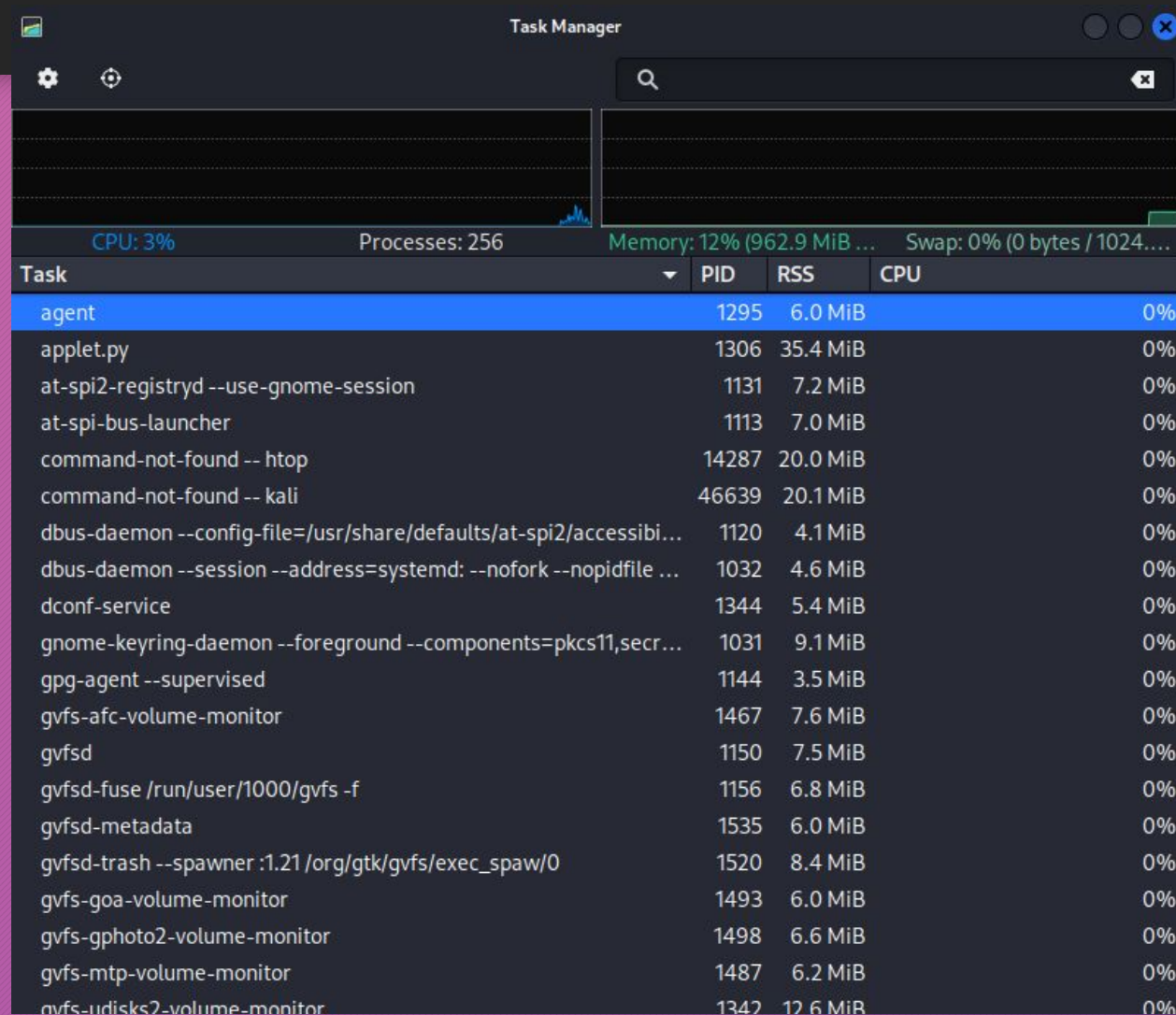
- Comando top
- El comando top es una herramienta en tiempo real para monitorizar los procesos que están en ejecución y muestra información detallada, como el uso de CPU y memoria

Identificación de los procesos del sistema

- PID: El ID del proceso.
- PR: Prioridad de programación de la tarea.
- NI: El valor nice de la tarea, que puede influir en su prioridad.
- S: Estado del proceso (R = en ejecución, S = en reposo).
- %CPU: Porcentaje de CPU que está usando el proceso.%MEM: Porcentaje de memoria RAM utilizado.
- TIME+: Tiempo total de CPU consumido por el proceso.
- COMMAND: El comando que originó el proceso.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1026	kali	9	-11	185080	24432	10420	S	22.5	0.3	0:24.72	pipewire
706	root	20	0	317408	8960	7552	S	13.6	0.1	0:33.64	vmtoolsd
1336	kali	20	0	220464	46236	29792	S	10.6	0.6	0:36.95	vmtoolsd
869	root	20	0	400708	118872	55280	S	5.6	1.5	1:47.67	Xorg
1252	kali	20	0	662602	66644	22712	S	2.2	0.5	0:02.17	xfsck-notify

Monitorizacion del sistema -entorno grafico



Secuencia de arranque del sistema. Demonios

1. Fuente de energía y señal al temporizador (timer) de la placa madre

- Cuando encendemos el ordenador, la fuente de energía (PSU) envía una señal de "Power OK" al temporizador de la placa madre. Esto le indica a la placa base que los niveles de energía son estables y que puede comenzar el proceso de arranque. El temporizador genera una señal de reinicio a la CPU, lo que inicia la ejecución de las instrucciones del **BIOS**.

Secuencia de arranque del sistema. Demonios

2. Bootstrap y arranque del hardware

- El BIOS (Basic Input/Output System) o el UEFI (Unified Extensible Firmware Interface) realiza el llamado **bootstrap**, que es el proceso de inicialización del hardware. El BIOS/UEFI ejecuta un **POST** (Power-On Self-Test), que verifica que el hardware esencial (memoria RAM, procesadores, dispositivos de almacenamiento, etc.) esté en buen estado de funcionamiento.

Secuencia de arranque del sistema. Demonios

3. El BIOS y el proceso de inicio

- Tras el POST, el **BIOS** busca un dispositivo de arranque, que normalmente es un disco duro, SSD o unidad USB, según la configuración de la secuencia de arranque. El BIOS localiza el sector de arranque del dispositivo, que contiene el **MBR** (Master Boot Record) o **GPT** (GUID Partition Table), dependiendo del esquema de partición utilizado.

Secuencia de arranque del sistema. Demonios

4. El MBR (Master Boot Record) y el GRUB

- El **MBR** se encuentra en el primer sector del disco de arranque y contiene el código necesario para cargar el gestor de arranque (bootloader). El MBR apunta al **GRUB** (GRand Unified Bootloader) o al gestor de arranque correspondiente, que se encarga de cargar el sistema operativo.
- **GRUB** es el gestor de arranque más común en distribuciones Linux. Su función es presentar un menú de opciones que permite al usuario elegir qué sistema operativo cargar (en el caso de sistemas de arranque múltiple). Si no se selecciona ningún sistema, el GRUB cargará automáticamente el kernel de Linux.

Secuencia de arranque del sistema. Demonios

5. Carga de la imagen del kernel de Linux (initrd)

- Después de seleccionar el sistema operativo en GRUB, este carga la **imagen del kernel de Linux** en la memoria. El **initrd** (initial RAM disk) es una imagen temporal que contiene un sistema de archivos mínimo necesario para montar el sistema de archivos raíz real. El kernel también detecta y configura el hardware, y comienza a inicializar los controladores necesarios.

Secuencia de arranque del sistema. Demonios

6. Carga de los servicios de inicio (SysVinit, systemd, upstart, etc.)

- Una vez que el kernel de Linux está cargado, comienza el proceso de **init** o **PID 1**. Este es el primer proceso de usuario que se ejecuta en el sistema y es responsable de iniciar todos los demás procesos y servicios necesarios para que el sistema funcione.

Secuencia de arranque del sistema. Demonios

7. Demonios en el Proceso de Arranque

- Los demonios son procesos que se ejecutan en segundo plano y que proporcionan servicios esenciales al sistema operativo. Estos procesos se inician automáticamente durante el arranque del sistema por el sistema de inicio
 - **cupsd**: Gestiona el servicio de impresión.
 - **sshd**: Demonio que proporciona el servicio SSH para conexiones remotas.
 - **cron**: Demonio que gestiona la programación de tareas automatizadas.
 - **networkd**: Gestiona las interfaces de red.
 - **httpd** o **nginx**: Demonios que proporcionan servicios web.

